react-native-urban-maps

Documento de arquitectura y decisiones técnicas

Versión 1.0 — 2026

Plataforma	React Native + Expo Dev Client
Renderizado	MapLibre GL (adapter nativo)
Datos de mapa	OpenStreetMap vía esquema OpenMapTiles
Tiles	Protomaps (.pmtiles) + Stadia Maps (fallback)
Estilo visual	Apple Maps (paleta cálida desaturada)
Base de datos	Supabase / PostgreSQL
Privacidad	3 niveles declarativos (ciudad / manzana / exacto)

1. Contexto y objetivo

La biblioteca react-native-urban-maps nace de la necesidad de mostrar ofertas de habitaciones en un mapa con distintos niveles de detalle y privacidad según el estado del listing. El objetivo es proporcionar una API declarativa que permita a la app consumidora renderizar mapas de calidad visual equivalente a Apple Maps, utilizando datos abiertos de OpenStreetMap.

1.1 Caso de uso principal

La app muestra listings de habitaciones con tres niveles de profundidad:

- Nivel 1 — búsqueda pública: cualquier usuario ve un pin aproximado en el distrito, sin revelar la calle.
- Nivel 2 — listing abierto: usuario interesado ve un pin en la manzana con un radio de incertidumbre de ~150 m.
- Nivel 3 — habitación asignada: el inquilino confirmado ve la dirección exacta y pin preciso.

> ✓ **Decisión**
> La lógica de qué nivel aplica a cada usuario es responsabilidad de la app consumidora (via RLS en Supabase). La biblioteca solo recibe el nivel como prop y renderiza en consecuencia.

2. Estilo visual — Apple Maps

El referente estético es Apple Maps. El estilo se implementa como un documento JSON de MapLibre GL con cinco pilares visuales:

2.1 Paleta cromática

Todos los valores están desaturados y tienen temperatura cálida. Nunca se usa blanco puro ni negro puro en ninguna capa del mapa.

Suelo base (background)	`#F5F2EB`	Beige arena cálido — nunca blanco
Edificios (building)	`#E8E2D5`	Beige más oscuro para dar volumen
Vías principales	`#FDFAF4`	Casi blanco pero con temperatura cálida
Vías secundarias	`#F7F3EA`	Un tono más oscuro que la principal
Parques (landuse park)	`#C8D9B0`	Verde salvia empolvado
Agua (water)	`#C5DCF0`	Azul cielo muy desaturado
Texto en mapa	`#4A4540`	Marrón oscuro, nunca negro

Halo de texto	`rgba(253,250,244,0.85)`	Mismo beige base con transparencia
Fondo noche	`#1C1C1E`	Grafito cálido — nunca negro puro
Texto noche	`#E8E4DC`	Crema cálido — nunca blanco puro

2.2 Patrones clave del estilo JSON

Interpolación por zoom

Todos los grosores de línea y tamaños de tipografía usan expresiones interpolate con zoom. Esto hace que el mapa 'despierte' progresivamente al acercarse, en lugar de que los elementos aparezcan bruscamente.

```
"line-width": ["interpolate", ["linear"], ["zoom"],
  10, 2,    // zoom 10 → 2px
  14, 8,    // zoom 14 → 8px
  18, 18    // zoom 18 → 18px
]
```

Sistema de capa doble en viales (case)

Cada tipo de vía tiene dos capas: la capa *-case (ligeramente más ancha, color beige muy suave) pintada primero, y encima la capa real en blanco cálido. Resultado: las carreteras tienen peso visual sin borde oscuro agresivo.

Text halo con temperatura cálida

En lugar de rgba(255,255,255,0.8), se usa rgba(253,250,244,0.85) — el mismo beige del fondo con transparencia. Las etiquetas se integran con el mapa en lugar de pegar sobre él.

2.3 Orden de capas

El array layers del estilo JSON debe seguir este orden para el aspecto correcto:

- background
- water
- landcover, landuse
- building (fill), building-outline
- road-*-case (capas dobles de viales)
- road-* (viales encima de sus cases)
- road-label (etiquetas de calles, siempre encima de los viales)
- poi (puntos de interés)
- place (etiquetas de ciudad, barrio, distrito)

3. Fuentes de datos — OpenStreetMap

3.1 Pipeline de datos

OSM no se consume directamente. Los datos se transforman en tiles vectoriales antes de que MapLibre pueda renderizarlos. El pipeline elegido es:

```
OSM planet.osm.pbf
    ↓  Planetiler (procesado ~2h para España)
output.pmtiles
    ↓  CDN / S3 con range requests
MapLibre GL (React Native)
```

> ✓ **Decisión**
> Protomaps + archivos .pmtiles es el stack elegido para producción. Un único archivo binario se sirve desde cualquier CDN sin servidor de tiles. Para desarrollo se usa Stadia Maps como fallback gestionado.

3.2 Proveedores evaluados

Protomaps	Gratis (self-hosted)	Nativo	ELEGIDO — producción
Stadia Maps	Free tier generoso	No real	Fallback en desarrollo
Maptiler Cloud	Free tier limitado	Parcial	Descartado
Self-hosted tileserver-gl	Infraestructura propia	Total	Descartado — complejidad alta

3.3 Esquema OpenMapTiles (OMT)

Todos los proveedores elegidos usan el esquema OpenMapTiles, que es lo que define las source-layer disponibles en el estilo JSON. Las capas más relevantes para uso urbano son:

`transportation`	class: motorway|trunk|primary|secondary|tertiary|minor|service|rail|subway · brunnel: bridge|tunnel · oneway · layer
`transportation_name`	name · name:es · ref (código autopista) · class
`building`	render_height (metros, para 3D) · render_min_height · class: apartments|commercial|industrial
`landuse`	class: park|cemetery|hospital|school|residential|industrial|commercial
`poi`	class: restaurant|cafe|hotel|hospital|subway|museum|shop · subclass · rank 1-10
`place`	class: city|town|suburb|neighbourhood · rank · population

`water`	class: ocean|lake|river|canal|swimming_pool

4. Base de datos — Schema y migraciones

4.1 Estado actual del schema

El schema de Supabase tiene ya la infraestructura básica de ciudades y lugares. La tabla listings tiene city_id como FK a cities (añadido recientemente), pero le faltan algunos campos críticos para los tres niveles del mapa.

`listings.city_id → cities`	**OK**	FK correcta. Permite join con centroid y bbox de ciudad.
`listings.city text`	**Redundante**	Texto libre duplicado. Risk de inconsistencia con city_id.
`listings.district text`	**Débil**	Sin FK a city_places. No permite join con centroide de barrio.
`listings.lat, lng`	**Básico**	Funciona pero sin índice geoespacial ni PostGIS geometry.
`listings.place_id → city_places`	**Falta**	Necesario para nivel 1: centroide del distrito exacto.
`listings.street, street_number`	**Falta**	Necesario para nivel 3: mostrar dirección completa al inquilino.
`listings.postal_code`	**Falta**	Útil para validación y geocoding.
`listings.osm_id`	**Opcional**	Permite reconciliar con OSM tras geocoding con Nominatim.
`cities.centroid jsonb`	**OK**	Parseable por la biblioteca. Suficiente para nivel 1.
`city_places.centroid jsonb`	**OK**	Usable una vez añadido place_id a listings.

4.2 Migración necesaria

Migración mínima para soportar los tres niveles del mapa:

```
ALTER TABLE public.listings
  ADD COLUMN place_id       text REFERENCES public.city_places(id),
  ADD COLUMN street         text,
  ADD COLUMN street_number  text,
  ADD COLUMN postal_code    text;

-- Índice para búsqueda por radio (aproximación válida para España)
CREATE INDEX listings_lat_lng_idx
  ON public.listings (lat, lng)
  WHERE lat IS NOT NULL;
```

⚠ **Atención**

PostGIS no es bloqueante para empezar. La búsqueda por radio con lat/lng plano tiene un error < 1% para la latitud de España y es suficiente para mostrar pins en el mapa. Se puede migrar a geography(Point, 4326) en una segunda fase.

4.3 Internacionalización de nombres

Para soportar nombres en múltiples idiomas (especialmente relevante en el País Vasco y Cataluña), las consultas de etiquetas del mapa deben usar coalesce:

```
"text-field": ["coalesce",
  ["get", "name:es"],
  ["get", "name"],
  ["get", "name:en"]
]
```

5. Arquitectura de la biblioteca

5.1 Capas del sistema

La biblioteca se organiza en cuatro capas con responsabilidades separadas. El principio rector es que solo el adapter nativo toca MapLibre — toda la lógica de negocio es TypeScript puro testeable sin montar un mapa.

1 — API pública	`<MapView />`, `<MiniMapView />`, `<ListingPin />`, `<ListingCluster />`, `<DistrictOverlay />`	Componentes que exporta la biblioteca. Lo único que ve la app consumidora.
2 — Core	`PrivacyEngine`, `ClusterEngine`, `StyleEngine`, `CameraController`	Lógica interna. TypeScript puro, sin dependencias nativas. 100% testeable.
3 — Adapter	`MapLibreAdapter`, `TileCache`, `SpriteRegistry`	Único punto de contacto con MapLibre. Intercambiable en el futuro.
4 — Nativo	`@maplibre/maplibre-react-native`, `Protomaps`, `Expo Location`	Dependencias externas. Requieren Expo Dev Client (no Expo Go).

5.2 Tipos TypeScript principales

```
// Nivel de privacidad — el contrato central
type PrivacyLevel = 1 | 2 | 3

// Datos en crudo que provienen de la DB
interface RawLocation {
  lat: number; lng: number
  cityId: string
  placeId?: string      // FK a city_places
  street?: string       // solo si level === 3
  streetNumber?: string
  postalCode?: string
}

// Contrato del pin — lo que recibe <MapView />
interface ListingPinData {
  id:            string
  privacyLevel:  PrivacyLevel
  location:      RawLocation
  price:         number
  currency?:     string     // default 'EUR'
  isHighlighted?: boolean
  metadata?:     Record<string, unknown>
}

// Props del componente raíz
interface MapViewProps {
```

```
  listings:        ListingPinData[]
  initialCity?:    { cityId: string; bbox?: BBox }
  styleMode?:      'day' | 'night' | 'auto'
  clustering?:     boolean
  onPinPress?:     (listing: ListingPinData) => void
  onBoundsChange?: (bounds: BBox) => void
  renderPin?:      (listing: ListingPinData) => React.ReactNode
  renderCluster?:  (count: number, listings: ListingPinData[]) =>
React.ReactNode
}
```

5.3 PrivacyEngine — lógica de ofuscación

El PrivacyEngine es la pieza más crítica de la biblioteca. Convierte un RawLocation con su PrivacyLevel en las coordenadas reales que ve MapLibre:

1	Centroide de city_places (barrio)	500 m	`"Malasaña, Madrid"`
2	Coords reales + offset determinístico ±150 m	150 m	`"Malasaña, Madrid"`
3	Coords exactas de DB	Sin radio	`"Calle Mayor 12, Madrid"`

> ✓ **Decisión**
> El offset de nivel 2 es DETERMINÍSTICO: usa las coordenadas reales como seed del hash. Mismo listing = siempre el mismo punto falso, independientemente de cuántas veces se re-renderice. Si se usara Math.random() el pin saltaría en cada render. Este patrón es el mismo que usa Airbnb.

5.4 Estructura de ficheros

```
react-native-urban-maps/
├── src/
│   ├── components/          # API pública
│   │   ├── MapView.tsx
│   │   ├── MiniMapView.tsx
│   │   ├── ListingPin.tsx
│   │   ├── ListingCluster.tsx
│   │   ├── DistrictOverlay.tsx
│   │   └── PrivacyCircle.tsx
│   ├── core/                # Lógica interna (puro TS)
│   │   ├── PrivacyEngine.ts
│   │   ├── ClusterEngine.ts
│   │   ├── CameraController.ts
│   │   └── StyleEngine.ts
│   ├── adapters/            # Único contacto con MapLibre
│   │   ├── MapLibreAdapter.tsx
│   │   ├── TileCache.ts
│   │   └── SpriteRegistry.ts
│   ├── styles/
```

```
│   │   ├── apple-day.json
│   │   ├── apple-night.json
│   │   └── sprites/
│   ├── hooks/
│   │   ├── useMapCamera.ts
│   │   ├── usePrivacyLocation.ts
│   │   └── useCluster.ts
│   └── types/index.ts
└── index.ts                # Barrel export
```

6. Ejemplo de uso desde la app

6.1 Pantalla de búsqueda

```
import { MapView } from 'react-native-urban-maps'

<MapView
  listings={listings.map(l => ({
    id: l.id,
    privacyLevel: 1,        // búsqueda pública → siempre nivel 1
    location: {
      lat: l.lat,
      lng: l.lng,
      cityId: l.city_id,
      placeId: l.place_id,  // para centroide del distrito
    },
    price: l.price,
  }))}
  initialCity={{ cityId: selectedCityId }}
  onPinPress={(listing) => router.push(`/listing/${listing.id}`)}
  onBoundsChange={(bounds) => refetchListings(bounds)}
/>
```

6.2 Pantalla de detalle del listing

```
import { MiniMapView } from 'react-native-urban-maps'

<MiniMapView
  listing={{
    id: listing.id,
    privacyLevel: isAssigned ? 3 : 2,  // app decide, biblioteca renderiza
    location: {
      lat: listing.lat,
      lng: listing.lng,
      cityId: listing.city_id,
      placeId: listing.place_id,
      street: isAssigned ? listing.street : undefined,
      streetNumber: isAssigned ? listing.street_number : undefined,
    },
    price: listing.price,
  }}
  height={220}
  interactive={false}
/>
```

7. Pendiente de definir

Los siguientes puntos están identificados pero aún no detallados:

- ClusterEngine — estrategia de agrupación de pins a distintos niveles de zoom (candidato: supercluster).
- Lazy loading de listings — implementación del callback onBoundsChange para refetch al mover el mapa.
- Inicialización de MapLibre con Expo Dev Client — configuración del plugin de Expo y prebuild.
- Pipeline de sprites — generación del sprite sheet de iconos POI estilo Apple.
- Geocoding con Nominatim — integración en el formulario de creación de listings para obtener lat/lng + osm_id desde dirección textual.
- Modo offline — estrategia de descarga selectiva de tiles por ciudad para uso sin conexión.

Documento generado automáticamente — 2026